

New Gadgetize

The $\times 2$ gadgetization recipe: design rationale and changes

local_mixing project

July 20, 2026

branch `sbg-gen-mix-clean`, commit `49c8cc02`

Abstract

New gadgetize is the name of the current production recipe for the $\times 2$ (two-share) gadgetization path of the CNOT-native backend (`sss --cnot --gadgetize --slice-zero-ccnot`). Relative to the previous plain gadgetization it adds two things: a *zero-slice preblock* S_1 that makes the circuit compute the target function only on one designated input slice (and, as a corollary, stops the inverse circuit from handing out C^{-1}), and a leaner, faster *randomization-gadget policy* that replaces the three-move RG set by the orthogonal basis $\{RG2, RG3\}$ at four times the rate. This note records the construction, the rationale, the guarantees (exact and measured-heuristic), the design alternatives that were considered and rejected, and the precise differences from previous versions.

1 Background: the $\times 2$ gadget path before this change

The gadget path transforms a reversible circuit C on n wires into a circuit on $2n$ wires: the first half (x , wires $0..n-1$) carries data, the second half (wires $n..2n-1$) is auxiliary. The emitted structure is

[bookend] [W_i encode] [SG/RG body] [decode] [bookend],

where the bookends are $\max(2n \ln n, 64)$ random g57 gates junk-targeting the second half; the W_i sweep (7 CNOTs per value) places every logical value v into a two-share pair (*share*, *pad*) with $v = \text{share} \oplus \text{pad}$, using the aux contents as masks; each logical g57 gate is realized by a four-fragment masking-safe SG (a four-cube ESOP of $B \vee \neg C$ over the four carriers, all fragments targeting one carrier of the output value); and the decode folds each value back onto its wire. The output contract (“GadgetLow”) is:

$$A(x, a) = (C(x), \text{junk}) \quad \text{for every aux value } a.$$

Two properties of this contract motivated the change:

1. **No slice conditioning.** The circuit computes C regardless of the aux input; there is no notion of a distinguished slice on which the functionality lives, unlike the feistel path’s slice modes.
2. **The inverse is free.** Because the contract holds for every a , the *inverse* circuit satisfies $G^{-1}(p, v) = (C^{-1}(p), \cdot)$ for every junk value v : anyone holding the (public, mixed) circuit can run it backwards and read C^{-1} off the low wires. The forward oracle gives C ; the naked gadget also gives away C^{-1} .

Additionally, the body’s randomization used three moves (RG1/RG2/RG3, uniform) at a rate of one RG per two SGs; §?? argues this set was redundant and the rate low.

2 The zero-slice preblock S_1

2.1 Construction

S_1 is prepended to the gadget (gate-list concatenation). It consists of G gates (default $10n$, flag `--slice-zero-ccnot-gates`) in *one uniform random order*, drawn from exactly two shapes, all with positive-polarity controls and all targets on the first half:

$$\underbrace{x_i \hat{=} a_j}_{\text{CNOT, } \approx 1/3} \qquad \underbrace{x_i \hat{=} x_j \wedge a_k}_{\text{CCNOT, } \approx 2/3}$$

(one control of each CCNOT on each half). Every gate reads at least one second-half wire, so on the all-zero slice $a = 0$ every gate is *individually* dead: $S_1(x, 0) = (x, 0)$ gate by gate, with no ordering or pairing argument. The second half passes through untouched (targets are confined to the first half), so the slice survives on its wires into the gadget.

For a fixed slice $a \neq 0$, each CCNOT collapses to a within- x CNOT (when its aux control is hot) and each CNOT to a constant flip, so the disturbance is a slice-indexed invertible *affine* map

$$M_a(x) = L_a x \oplus t_a.$$

The composite circuit therefore computes

$$A(x, 0) = (C(x), J), \quad A(x, a) = (C(M_a(x)), J') \quad (a \neq 0),$$

and since C is a permutation, a wrong slice reproduces C at x exactly when M_a fixes x .

2.2 Guarantees

Superseded 2026-07-25; see docs/BAND_HARDENING §3. The construction below is the original one, whose guarantee was exact on slices firing no CCNOT and heuristic elsewhere. It has since been rebuilt around a target/control split whose order-free algebra makes “only the all-zero slice is fixed” a **theorem for every slice**, checked outright on the emitted gates by a rank test—and the slice was widened to include the product-share band. The historical statement follows.

Exact (original). The CNOT pin multiset (a random permutation seed plus random extras) was resampled until its target-by-control parity matrix $\mathbf{C}$ was invertible over $\text{GF}(2)$. Consequently, on any slice that fires no CCNOT the disturbance is the pure translation $x \oplus \mathbf{C}a \neq x$: every such slice is disturbed at every input, provably. (Invertibility also guarantees every aux wire is read.)

Heuristic, measured (original). On slices firing CCNOTs, “no wrong slice is fixed” was not a theorem: it fails if the fired transvection subsequence composes to the identity *and* the interleaving-conjugated CNOT translations cancel. This was measured directly (exhaustive slice check per sampled preblock, gate budget $10n$):

n	3	4	5	6	8
preblocks with a fixed wrong slice	$\approx 4 \cdot 10^{-3}$	$4 \cdot 10^{-4}$	$2 \cdot 10^{-5}$	$4 \cdot 10^{-5}$	0
trials	2,000	50,000	50,000	50,000	50,000

The rate decayed rapidly in n and was negligible at production widths. The current construction removes the residual entirely: exactness now holds at every n , including the toy widths where this table shows failures, and the test suite asserts it exhaustively rather than pinning a counterexample.

Inverse protection. In the inverse circuit A^{-1} (the reversed gate list; every gate is an involution), S_1^{-1} runs *last* and fires on the gadget’s generically nonzero mask residue, junking the low half. This closes the free- C^{-1} hole of §1: a bare gadget’s inverse returns C^{-1} for any junk input; with S_1 it does not. This is regression-tested (`slice_block_stops_the_inverse_from_revealing_c_inverse`).

2.3 Why this vocabulary

The preblock is pure positive-polarity CNOT/Toffoli material: indistinguishable in gate shapes from ordinary mixed-circuit content, with no complemented conjunctions and no polarity pattern encoding a slice. Two contrasts drove this choice:

- The feistel path’s random-slice mode (mode 3) encodes its public slice *in the clear* in the control polarities of its core OR-gates. With the slice fixed to the all-zero vector and positive controls throughout, there is nothing to read off.
- The legacy r57-only vocabulary ($a \hat{=} b \vee \neg c$) cannot express a gate that both reads an x wire and is dead on a slice; its slice constructions needed commuting-pair cancellation tricks. The CCNOT shape $x_i \hat{=} x_j \wedge a_k$ achieves per-gate deadness and x -fanout simultaneously.

2.4 Alternatives considered and rejected

Contiguous cnot certificate block (rigorous uniqueness). Placing all CNOTs in one contiguous run at an end of the block makes uniqueness a theorem: the run commutes into $x \oplus \mathbf{C}a$, the CCNOT soup is translation-free per slice, and $M_a = \text{id}$ forces $\mathbf{C}a = 0$, i.e. $a = 0$. This was implemented (in one-end and symmetric two-end variants) and *rejected by design decision*: the uniform random order was preferred — the CCNOTs do not commute, so the order itself is part of the randomness, and bunched CNOT runs are structure the artifact should not carry. The cost of that choice is exactly the heuristic table above.

Two-slice sandwich $S_1 \parallel G \parallel S_2$. A closing slice block was fully built (with the gadget replumbed to decode onto the high wires so S_2 ’s first-half targets would only touch junk) and then *reverted*: it belongs to the feistel-style sandwich construction (where an N -step moves the answer to the second half and a random D junks the first), not to the general $\times 2$ gadget mechanism, whose answer exits on the low wires — there, a closing S_2 ruins correctness. What was kept from the detour is the inverse-direction analysis and its test.

3 The new RG policy

3.1 The moves

With two-share pairs $i = (i_0, i_1)$ and $j = (j_0, j_1)$:

	gates	effect	role
RG1	6 CNOTs	values swap wire-sets	<i>dropped</i>
RG2	3 CNOTs (swap $i_0 \leftrightarrow j_0$, re-pair crosswise)	pairing re-dealt, values intact	sole re-pairer
RG3	2 CNOTs ($i_0 \hat{=} r$, $i_1 \hat{=} r$, foreign r)	masks refreshed, value intact	sole mask injector

All moves are linear and *gate-locally non-complete*: no physical gate ever reads both carriers of one logical value, so first-order prefix masking is preserved — this invariant is unchanged from the previous version and regression-tested.

3.2 Why $\{RG2, RG3\}$ is a basis and RG1 is redundant

RG2 and RG3 are orthogonal: RG2 changes *which wires are paired* (compositions of RG2 only *permute* wire contents — each application is a content swap), while RG3 is the only move that *recombines mask values* (XORing a foreign carrier into both members of a pair). Neither can do the other’s job, and together they generate every sharing configuration.

RG1 is a composite: two generalized RG2s (swap $i_0 \leftrightarrow j_0$, then $i_1 \leftrightarrow j_1$) reproduce its value-swap at identical CNOT cost, and the one thing the swap route lacks — RG1’s mask recombination (its outputs are XOR mixtures such as $(b \oplus c, b \oplus d, a, b)$, versus the pure content permutation (d, c, b, a) of the two-swap route) — is precisely RG3’s job. RG1 earned its former place by packaging value-swap-plus-mask-stir in 6 CNOTs; with RG3 present at a higher rate it is redundant, and dropping it simplifies the move algebra.

3.3 Rate

Previously: one uniform RG1/RG2/RG3 draw after every second SG (0.5 RGs per SG, ≈ 1.8 CNOTs per SG on average). Now: **--rg-frequency** independent uniform RG2/RG3 draws *between every two consecutive SGs* (none after the last, immediately before the decode). The default of 2 gives 2 RGs per gap = $4\times$ the move rate, ≈ 5 CNOTs per SG ($2 \times \frac{3+2}{2}$) $\approx 2.7\times$ the CNOT volume. The sharing now drifts faster than any single SG’s four-fragment footprint. Note the flag’s meaning *changed* for this path (it was “one RG every K SGs”); the feistel and legacy paths keep the old meaning and the full RG1/RG2/RG3 set.

One acknowledged trade-off: RG2 is a contiguous 3-CNOT swap, so the raw (unmixed) artifact contains more recognizable swap blocks than RG1’s interleaved networks produced. The mixer dissolves these immediately; the previous version’s material was prettier pre-mixing, not stronger post-mixing.

4 Differences from previous versions: summary

	plain gadgetize (previous)	new gadgetize (49c8cc02)
invocation	<code>sss --cnot --gadgetize</code>	<code>sss --cnot --gadgetize --slice-zero-ccnot</code>
contract	$C(x)$ on low wires for <i>every</i> aux input	$C(x)$ on low wires <i>only</i> on the $a=0$ slice; $C(M_a(x))$ off-slice
inverse circuit	returns C^{-1} for any junk input	low half junked by S_1^{-1}
preblock	none	$10n$ -gate CNOT/CCNOT slice block, uniform order, invertible parity matrix
RG set	RG1, RG2, RG3 (uniform)	RG2, RG3 only
RG rate	one per 2 SGs (≈ 1.8 CNOT/SG)	two per SG gap (≈ 5 CNOT/SG)
<code>--rg-frequency</code>	SGs per RG	RGs per SG gap (this path only)
verification	GadgetLow over random aux	GadgetLow pinned to the zero slice, at transform and every round
size ($n=8$, 40 gates)	≈ 560 gates	≈ 670 gates

Relation to the feistel slice modes (for orientation): the feistel path offers a linear zero-slice mode ($x \oplus A(y \vee z)$), a nonlinear hardcoded zero-slice mode, and a random *public* slice mode ($R^{-1}BR$, uniquely fixing by construction but carrying the slice in its gate polarities). New gadgetize is the $\times 2$ analogue of a zero-slice mode with mixing-native vocabulary: per-gate deadness, no polarity signature, exact uniqueness on the CCNOT-silent subclass and measured heuristic uniqueness elsewhere.

5 Open questions

1. **S_1 randomizer strength.** Per slice, S_1 is a shallow affine map at the $10n$ budget (each first-half wire written ≈ 10 times; no nonlinearity in x , since $x \wedge x$ controls are excluded by the gate-shape spec). Whether this matters for post-mixing inversion resistance is open; the knob is `--slice-zero-ccnot-gates`, and a planned measurement is avalanche/dislocation of an isolated S_1 versus budget at production n .
2. **Feistel port.** Neither the RG policy nor the slice block has been ported to the feistel path. Note the invertibility certificate does not transfer as-is: the feistel aux block has $2n$ wires, so the parity matrix would be $n \times 2n$ and never injective; a feistel variant needs mode-3-style pairing or a purely heuristic argument.
3. **Deployment.** Not yet built on the .242 server.

Provenance. Code: `slice_zero_ccnot_preblock`, `gadgetize_with_slice_zero_ccnot`, and the RG loop in `src/replace/gadgets.rs`; driver in `src/replace/main_mix_cnot.rs`. Tests:

four slice-block unit tests plus the zero-round driver case; full suite 139 passing at commit 49c8cc02. Companion quick-reference: `docs/NEW_GADGETIZE.md`.